

Stage 5 Report

学号: 2022010776

姓名: 张瀚宸

实验内容

step 9

这阶段的主要难点就是参数的传递。为了实现高效的参数传递,我在

`backend/reg/bruteregalloc.py` 中对 `Call` 指令做了特判,实现了相应的寄存器分配。

这需要十分小心,因为此时寄存器分配器中原本的状态会遭到不小的破坏,因此需要谨慎应对可能存在的一些冲突:比如我需要用 `A0`

来传递参数,但是 `A0` 可能已经被分配给了其他变量,这时候就需要将 `A0` 的绑定关系解除,然后再将 `A0` 分配给参数。

为了解决在寄存器分配器中传入参数的绑定问题,我构造了虚拟指令 `Load Params` 来对应将 `Temp` 与 `Reg` 相绑定的过程。

`Load Params` 在中间语言的定义就是稳定出现在函数开头作为第一条指令,不进行任何实际运算,仅占位。

除此之外,超过8个参数的函数调用需要使用栈来传递参数,因此需要对栈进行相应的操作,这也存在一定的难度。因此我对 `riscvasmemitter.py` 进行了一定的修改,实现了参数的栈传递。

值得一提的是,在写好了初版以后,我对寄存器分配过程中的局促和仓皇感到十分不安,因此在后续进行了一次优化。

我没有学习复杂的寄存器分配算法,仅仅是力所能及的一点改进。为了减少内存读写,在进行寄存器分配时,我规定优先使用 `Callee Save` 寄存器。

同时,我实现了一个**预约机制**,就是对于特定的 `Temp`

,如果他确信自己在不久的将来需要出现在某个特定的寄存器之中,例如进行参数传递时,那么他将对这个寄存器进行预定,此时如果别的 `Temp`

想要获得一个寄存器,那将优先绕开被预定的寄存器。而对于这个 `Temp` 本身,在机会出现的之后将会直接被保存在这个寄存器里。

这里需要注意的是同一个寄存器可能会被多次预约,例如一个过程里有多个函数调用,那就会有許多 `Temp` 预约到 `A0-A7`,因此需要实现一个预约的队列。

思考题

step 9

1. 我更倾向于一条独立的指令。这样的好处是更简洁明了地表达了函数调用,而且给了后端更大的自由度,也给了中端更自由的优化空间。对于二者的优劣,我大致是这样认为的

	单条	多条
优势	简洁明了，前端好写，自由度高	前后端后端都好实现，方便
劣势	需要做不少额外处理，后端写起来更麻烦	中间表示会变复杂，优化空间会小一些

- 可能因为这样可以减少内存读写。对于Caller来讲，他可以把一些没那么重要的东西，也就是之后不用了的东西优先存在Caller Save寄存器上，而对于Callee来讲，如果自己没有用到某些Callee Save Reg，那就可以不用负责保存和恢复。整体来讲可以减少一些不必要的内存读写。对于为什么 ra 是Caller Save，不难想到这是因为函数调用的时候需要用到 ra 来保存返回地址，这个值在调用发生的时候就会被覆盖。而这时 ra 还存着自己的返回地址呢，所以当然自己要好好保存。